



Machine Learning for Petroleum Engineers Using Python

Módulo 7 · Clase 1

Lo Que Antes Necesitaba un Programador



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Dónde Estamos: Cambia la Pregunta

✓ Hasta aquí:

- ✓ Aprendieron a **plantear** un problema de datos
- ✓ A **medir** si un modelo sirve, contra un rival simple
- ✓ A **desconfiar** del dato antes que del modelo
- ✓ Y a entregar una **banda**, no un número solo

▶▶ Desde hoy:

- Ya no preguntamos «¿qué modelo uso?»
- Preguntamos «¿cómo construyo la **herramienta que hace falta?**»
- Y la construimos **dirigiendo a un agente**
- ★ Hoy: **tres demos más, tres ejercicios suyos**, y una app con link al final

Todo eso es criterio de ingeniería. No caduca.

Los ejercicios no tienen solución escrita en ninguna parte. Se resuelven acá, en vivo — así que pregunten sin pena.

El Proyecto del Módulo, Desde Ya

space1pt

Una aplicación que resuelva un problema real de su trabajo, construida dirigiendo a un agente, con su link funcionando.

Se entrega con los **prompts** que usaron, **un error del agente que ustedes cazaron**, y qué **decisión** habilita la herramienta.

space1pt

-  **Equipos de hasta 3 personas**, máximo. Solo también vale.
-  **Entrega: hasta el jueves de la próxima semana.** No se corre.
-  **Esta semana** se introducen las herramientas **reales** de producción de ML en la industria: agentes, servir un modelo, apps. Cada clase le suma una pieza a su proyecto.

space1pt **Tarea de hoy mismo: elijan equipo y problema.**

Ustedes Ya Dirigen Trabajo Técnico

Cuando le encargan un trabajo a un contratista, ya hacen exactamente lo que vamos a hacer hoy con un agente:

1 • Especifican

No dicen «arregle el pozo». Dicen qué, con qué materiales, para cuándo y cómo se sabe que quedó bien.

2 • Reciben y revisan

No firman el acta sin mirar. Revisan contra lo que pidieron, no contra lo que les entregaron.

3 • Responden

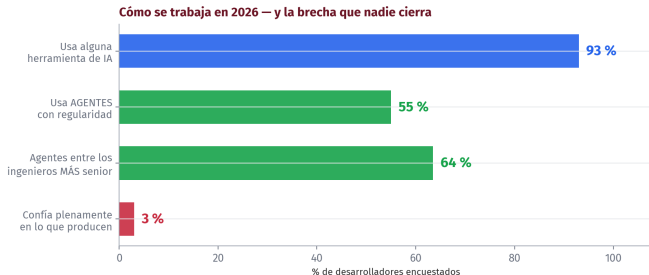
Si el trabajo sale mal, la firma es de ustedes. El contratista no va a la reunión.

Un agente de código es un contratista muy rápido, muy barato y muy seguro de sí mismo. Todo lo que saben de dirigir contratistas aplica — incluida la parte de que la firma sigue siendo suya.

Esto Ya Pasó

Cómo se escribe código en 2026, con números
o – 18 min

La Foto de 2026



El **93 %** lo usa, y solo el **3 %** le cree del todo. Las dos cosas son ciertas a la vez — y esa brecha es exactamente la habilidad que se paga: usarlo **sin** creerle a ciegas.

Lo que No Dicen los Titulares

Todo el mundo lo usa. Casi nadie le cree del todo. Las dos cosas están medidas:

LA CONFIANZA

Solo el **3 %** confía plenamente en lo que produce.

El **46 %** desconfía de su precisión, contra un 33 % que confía.

LA LETRA CHICA

Frustración #1, con el **66 %**: soluciones «*ca-si correctas, pero no del todo*».

Y el **45 %** dice que depurarlas tarda **más** que escribir a mano.

«Casi correcto, pero no» es el problema central de esta clase — y es justo donde hace falta alguien que sepa del tema.

El Dato que le Conviene a Esta Sala

Los ingenieros más senior son los que más usan agentes: 63,5 % contra 55 % del promedio.

No es una herramienta para el que no sabe. Es una herramienta que **rinde más cuanto más criterio tiene quien la usa** — porque el que sabe reconoce al instante cuándo la respuesta está mal.

Y quien usa agentes tiene **el doble de probabilidad** de estar entusiasmado; quien no los usa, el doble de escéptico. La frustración #1, con el 66 %: «*soluciones casi correctas, pero no del todo*». **El**

escepticismo, en esto, se cura usándolo. Por eso hoy se usa.

El Cuello de Botella se Movi 



El ancho es el tiempo. Lo rojo es donde se atasca el trabajo.

Escribir c digo dej  de ser lo caro. Lo caro ahora es **verificar**. Y verificar exige saber del negocio — que es lo que ustedes traen.

Y Por Eso Ustedes Valen Más, No Menos

Lo que se volvió barato es *escribir*. Lo que sigue siendo escaso es saber si el resultado tiene sentido.

Un agente escribe en treinta segundos el código que ajusta una curva de declinación. Lo que no puede saber es que ese campo tuvo un *workover* en 2018, que el medidor estuvo descalibrado seis meses, o que ese pozo no debería estar en la muestra.

Eso lo saben ustedes. Y hasta hoy, convertir ese conocimiento en una herramienta significaba esperar a que alguien de sistemas tuviera tiempo. ***Eso es lo que se acabó. De eso se trata este módulo.***

Acotar el Problema

LO QUE SÍ VAMOS A HACER HOY

- ✓ Gráficas que se entienden solas
- ✓ Dirigir un agente con encargos bien escritos
- ✓ **Verificar** lo que devuelve
- ✓ Una app con link, hecha por ustedes

LO QUE NO

- ✗ Convertirlos en programadores
- ✗ Memorizar sintaxis de Python
- ✗ Instalar nada: todo corre en Colab
- ✗ Modelos nuevos — hoy se **construye**, no se predice

Regla de la casa: si algo no se entiende, es culpa del material, no de ustedes. Pregunten en el momento.

Las Herramientas

El panorama de 2026, y las cinco que usamos hoy

18 – 40 min

Colab, Desde Cero

Para quien nunca lo ha abierto — y de recorderis para el resto:

- Un **cuaderno** es una página hecha de **celdas**: unas tienen texto, otras tienen código.
- Una celda de código se ejecuta con **Shift + Enter** (o el botón ▶). El resultado aparece justo debajo.
- Todo corre en un computador de Google, no en el suyo. Por eso **no hay que instalar nada**.
- El cuaderno se guarda solo en su **Drive**. Si la sesión se desconecta, el texto queda — solo hay que volver a correr las celdas.

Regla práctica de hoy: las celdas se corren en orden, de arriba hacia abajo. Si algo falla, casi siempre es una celda de más arriba que no se corrió.

Dónde Vive el Agente

El agente de hoy es el **Gemini de Colab**. Así se le habla:

1. Abran el cuaderno de la clase en `colab.research.google.com`.
2. Arriba a la derecha está el botón **Gemini** (el ícono ). Se abre un panel de chat al costado.
3. Ahí se **pega el encargo**. El agente propone código; se revisa y con **«insertar»** pasa al cuaderno como celda nueva.
4. Si una celda falla, aparece el botón **«explicar error»**: úsenlo — es la manera más rápida de aprender qué pasó.

SI EL PANEL NO LES APARECE

Requiere cuenta de 18+ y varía por región. No es problema: armen el encargo igual, y lo corremos en el proyector — el aprendizaje de hoy está en **escribir el encargo**, no en apretar el botón.

El Panorama a 2026

Estos son los agentes de código que dominan el mercado. Los nombres van a cambiar; **la manera de dirigirlos, no** — y eso es lo que se aprende hoy:



GitHub Copilot

GitHub Copilot
dentro del editor



OpenAI Codex
agente en la nube



Cursor
un editor entero con IA



Gemini
el de Google

ANTHROPIC

Claude Code
agente en la terminal



Colab + Gemini
el nuestro: ya lo tienen

Todos hacen lo mismo por dentro. Usamos el de Colab: cero instalación, cero pago.

Las Cinco de Hoy, y Qué Hace Cada Una



- **Colab** — el taller: un cuaderno de Python en el navegador, gratis, sin instalar nada.
- **Gemini** — el contratista: el agente que vive dentro de Colab (el ícono ✎ del panel). A él le vamos a dictar los encargos.
- **pandas** — la mesa de trabajo: carga los datos y los deja en *tablas* que se manipulan con código.
- **seaborn** — el dibujante: gráficas con las decisiones de diseño ya tomadas. Además trae los datos de práctica de hoy.
- **Gradio** — la vitrina: convierte una función en una página web con controles y un **link** para compartir.

Las cinco ya están en Colab. Hoy no se instala absolutamente nada.

pandas, la Mesa de Trabajo

Casi todo lo de hoy pasa por un **DataFrame**. Es solo esto:

Una tabla con filas y columnas — como una hoja de cálculo — pero que se maneja con código en vez de con el mouse.

Cada fila es un caso (un viaje, un vehículo, un diamante). Cada columna es un dato de ese caso, con su tipo: número, texto o fecha.

Las tres operaciones que van a ver todo el día:

- `taxis.head()` — muestra las primeras filas, para mirar qué hay
- `taxis["hora"]` — una columna, por su nombre
- `taxis.groupby("hora").size()` — cuenta casos por grupo

Qué Es un Agente (y Qué No)

Tres cosas distintas que la gente llama «la IA»:

- **Autocompletado.** Sugiere el final de la línea. Entiende el patrón, no el problema.
- **Asistente.** Uno pregunta, él contesta con código para copiar. *Él no ve el resultado.*
- **Agente.** Escribe el código, **lo ejecuta, mira lo que salió** — incluidos los errores — y **corrige solo.** Repite hasta que corre.

La diferencia no es que adivine mejor: es que itera.

Y por eso, cuando se equivoca, se equivoca con más convicción: ya probó, ya corrigió, y entrega algo que *corre*. Que corra no significa que esté bien — esa es la lección grande de hoy.

Los Cuatro Fundamentos de un Encargo

Las cuatro cosas que se le dicen a cualquiera a quien uno le encarga un trabajo:

1 • OBJETIVO

Qué pregunta quiero responder. **No** qué gráfico quiero: qué quiero *saber*.

3 • RESTRICCIONES

Qué herramientas usar, en qué idioma, qué **NO** hacer. Sin esto, inventa lo que le parezca.

2 • CONTEXTO

Qué datos hay, cómo se llaman las columnas, en qué unidades, para quién es el resultado.

4 • CRITERIO DE ACEPTACIÓN

Cómo sabemos que quedó bien. El que casi nadie escribe, y el que más cambia el resultado.

El cuarto convierte un pedido en un encargo de ingeniería. En los ejercicios de hoy, ustedes lo escriben.

La Plantilla que se Llevan

Los cuatro fundamentos, listos para llenar. Sirve para una gráfica, un análisis o una app entera — y es la que van a usar en los tres ejercicios de hoy:

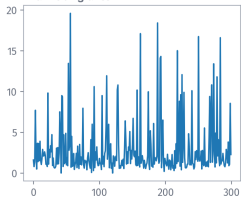
CONTEXTO: tengo [qué datos], con columnas [cuáles] en [qué unidades]. Es para [quién / qué reunión].
OBJETIVO: quiero responder [qué pregunta].
RESTRICCIONES: - usa [qué librerías] - [qué NO hacer] - en español, ejes con unidades
CRITERIO DE ACEPTACIÓN: - [qué tiene que verse / cumplirse] - avísame si [qué problema podría tener el dato]

El último renglón — «avísame si...» — convierte al agente en un aliado para encontrar problemas, en vez de alguien que los tapa.

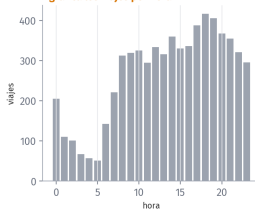
La Escalera: el Mismo Agente, Tres Pedidos

La calidad de la salida es la calidad de la especificación

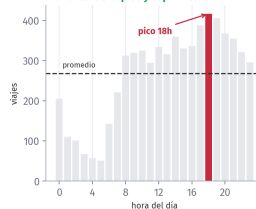
«hazme un gráfico»



«grafica los viajes por hora»



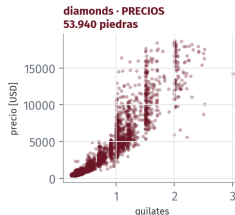
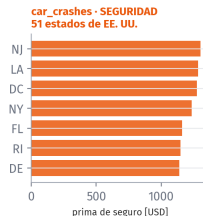
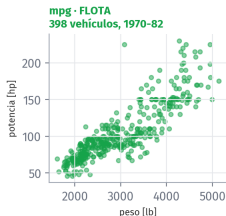
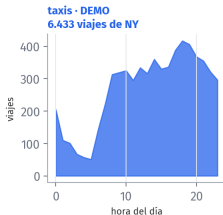
«...marcando el pico y el promedio»



«hazme un gráfico» → «grafica los viajes por hora» → «...marcando el pico y el promedio». **La calidad de la salida es la calidad de la especificación.**

Los Cuatro Datos de Hoy

Los cuatro datos de hoy: uno para las demos, tres para ustedes



Todos vienen **dentro de seaborn**: se cargan con una línea. Con taxis hago yo las demos; mpg, car_crashes y diamonds son **de ustedes**. A propósito no hay ninguno de petróleo: la habilidad es la misma, y el proyecto final es con SU dato.

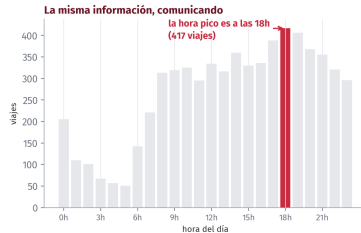
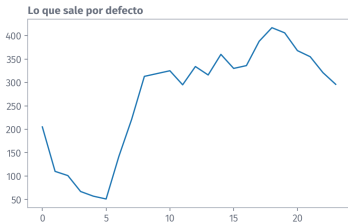
Demo 1 • Reto 1

Una gráfica que no hay que explicar

40 – 60 min

Demo · Los Mismos Datos, Dos Veces

Los mismos datos, los mismos números. Solo cambia si se entiende.



Mismos datos, mismos números, misma librería. A la izquierda hay que explicar; a la derecha, la respuesta salta sola. **Un gráfico que hay que explicar hablando es un gráfico que falló.**

Demo · Las Cinco Decisiones

No fue magia. Fueron cinco decisiones, y todas se le pueden **pedir** a un agente:

1. **Barras en vez de línea** — las horas son categorías separadas, no un continuo.
2. **Un solo color con sentido** — gris todo, rojo lo que importa. El color señala, no decora.
3. **Los ejes dicen qué son**, en el idioma del que lee.
4. **El hallazgo, escrito sobre la figura** — la conclusión va en la lámina, no en la boca del presentador.
5. **Se quitó todo lo demás** — sin marco, sin cuadrícula de más, sin decimales que nadie pidió.

Ninguna es programación. Las cinco son criterio de comunicación — por eso las deciden ustedes, no el agente.

Demo · El Encargo que le Di

Tengo un DataFrame 'taxis' de seaborn con viajes de taxi de Nueva York.
Columnas: pickup y dropoff (fecha y hora), distance, fare, tip, total, payment, pickup_{zone}, pickup_{borough}.
OBJETIVO: quiero saber a qué hora del día hay más demanda de taxis.
RESTRICCIONES: - usa seaborn - va en un reporte para gerencia: tiene que entenderse sin que yo lo explique - ejes con nombre y unidades, en español
CRITERIO DE ACEPTACIÓN: - la hora pico marcada y legible de un vistazo - avísame si hay horas sin ningún viaje registrado

Fíjense dónde está cada fundamento. Y fíjense en lo que NO dice: no dice qué tipo de gráfico hacer — esa decisión se la dejé, y la revisé al recibir.

🔧 Reto 1 • La Flota

🕒 12 min

La gerencia de flota pregunta: ¿los vehículos mejoraron su consumo a lo largo de esos años — y cuánto?

Columnas que les sirven: mpg (millas por galón: más es mejor), model_year, origin, weight.

Carguen el dato con `mpg = sns.load_dataset("mpg")` y armen **su propio prompt** con los cuatro fundamentos.

Cómo saber si van bien (preguntas, no respuestas):

- ? ¿Su gráfica se entiende sin que ustedes hablen?
- ? ¿El hallazgo está *escrito* en la figura, con número?
- ? Este dataset tiene **valores faltantes** en una columna. ¿El agente les avisó, o se los tragó sin decir nada?

Reto 1 • Puesta en Común

Antes de ver ninguna «respuesta»: comparen entre mesas.

- ¿Qué tipo de gráfico eligió el agente de cada grupo — línea, barras, cajas? ¿Quién tomó esa decisión: ustedes en el prompt, o él por defecto?
- ¿A alguien le **avisó** de los valores faltantes sin que se lo pidiera? Lean ese prompt en voz alta: ¿qué tenía distinto?
- El número final — ¿cuánto mejoró el consumo? ¿Todas las mesas sacaron **el mismo** número? Si no: ¿por qué no?
- Miren el prompt del compañero de al lado: ¿qué le copiarían?

Dos prompts distintos dan dos resultados distintos sobre el mismo dato. Esa es la lección — y por eso el prompt se escribe con cuidado.

Demo 2 • Reto 2

Una pregunta de operación

60 – 76 min

Demo · Ahora una Pregunta que Decide Turnos

¿Cuándo conviene tener más gente en la calle? — la pregunta de cualquier jefe de operaciones. Así se la encargué:

CONTEXTO: el mismo DataFrame 'taxis'. Ya tiene las columnas hora (0-23) y día (0=lunes ... 6=domingo).

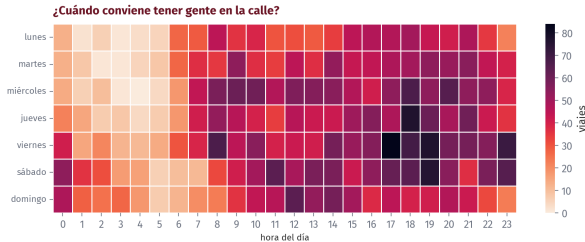
OBJETIVO: saber en qué combinaciones de día y hora se concentra la demanda, para decidir turnos.

RESTRICCIONES: - un mapa de calor con seaborn: días en filas, horas en columnas
- días en español, en orden de lunes a domingo - nada de escalas rojo-verde (hay daltónicos)

CRITERIO DE ACEPTACIÓN: - que se pueda señalar con el dedo el peor momento - dime cuál es y cuántos viajes tiene

La restricción del rojo-verde no es estética: es que el gráfico lo pueda leer toda la sala. Ese tipo de restricción solo la pone quien conoce a su público.

Demo · Lo que Devolvió



El momento crítico es el **viernes a las 17h**. Con eso ya se decide un turno — y la decisión no la tomó el agente: la tomó la **pregunta**.

🔧 Reto 2 · Seguridad Vial

🕒 12 min

El área de seguridad pregunta: **¿los accidentes van de la mano del alcohol? ¿Y en qué estados habría que priorizar una campaña?**

Columnas que les sirven: `total` (accidentados por cada 100 millones de millas), `alcohol`, `speeding`, `abbrev` (el estado).

```
crashes = sns.load_dataset("car_crashes")
```

 — y su prompt, con los cuatro fundamentos.

Cómo saber si van bien:

- ? ¿La relación entre alcohol y accidentes se ve, o hay que crearla de palabra?
- ? ¿Pueden señalar con el dedo **tres estados** para la campaña, y defender por qué esos?
- ? Pregunta brava: que dos cosas vayan juntas, ¿prueba que una causa la otra? ¿Qué le contestarían al gerente que lo afirme?

Reto 2 · Puesta en Común

Otra vez entre mesas, tres preguntas:

- 💬 ¿Cómo mostraron la **relación**: puntos, ranking, mapa de calor? ¿Se ve, o hay que creerla de palabra?
- 💬 Los **tres estados** para la campaña: ¿coinciden entre mesas? Si dos mesas eligieron distinto con el mismo dato, ¿qué criterio usó cada una?
- 💬 La brava: alcohol y accidentes van juntos. ¿Eso **prueba** que uno causa el otro? ¿Qué más podría explicar la relación?

Correlación no es causa — y un agente les va a entregar la correlación con total confianza. La pregunta de causa la hace el ingeniero, no la herramienta.

Demo 3 · Reto 3

Donde el agente falla con confianza

76 – 98 min

Demo · Ahora la Parte Incómoda

Hasta acá todo salió bien, y esa es justamente la trampa. Vamos a hacer una pregunta perfectamente razonable sobre los mismos taxis:

«¿Los pasajeros que pagan en efectivo dejan menos propina que los que pagan con tarjeta?»

Es el tipo de pregunta con la que se decide una política de medios de pago. Tiene consecuencias en dinero.

El agente la responde en veinte segundos, con un gráfico impecable. Y antes de mostrarlo, una pregunta para la sala: ¿qué le pedirían ustedes ANTES de creerle?

Demo · Un Análisis Impecable



«Quien paga en efectivo nunca deja propina»

Y es falso

De los 1812 viajes en efectivo, exactamente 0 tienen propina.

No es que no la dejen: el taxímetro solo registra la propina cuando va en la tarjeta. La de efectivo va al bolsillo del conductor y nunca entra al sistema.

Un cero que no es un cero: es una **AUSENCIA**

El cálculo está bien hecho. El gráfico está bien hecho. La conclusión es **falsa**: la propina en efectivo va al bolsillo del conductor y **nunca entra al taxímetro**. Un cero que no es un cero: es una **ausencia**.

Demo · Esto Ya lo Vivieron

Es el mismo problema del módulo del agua: campos con **cero agua producida** durante veinticinco años. No es que no produjeran — es que el regulador no la exigía hasta el año 2000.

Distinto dominio, distinto dato, distinto país — el mismo error.

Y en los dos casos lo que lo detectó no fue una técnica: fue alguien que conocía el negocio y dijo «*esto no puede ser*».

El agente no tiene esa alarma. Ustedes sí. Y se puede convertir en método:

Antes de concluir nada de ‘taxis’: revisión de calidad. Filas totales y faltantes; mín/máx/ceros por columna numérica; y DIME si algún grupo tiene SIEMPRE el mismo valor (suele ser artefacto del registro). Lista lo raro, aunque no estés seguro.

🔧 Reto 3 · Los Diamantes

🕒 12 min

Van a analizar precios de diamantes... pero todavía no. Primero: **córranle su revisión de calidad**. Este dataset esconde **al menos un valor físicamente imposible** — y más de uno.

Columnas: carat (quilates), cut, price [USD], y x, y, z — las **dimensiones físicas en milímetros**.

```
diamonds = sns.load_dataset("diamonds")
```

Cómo saber si van bien:

- ? ¿Qué significa físicamente que una dimensión valga **cero**? ¿Cuántas filas así hay?
- ? Miren el **máximo** de cada dimensión. ¿Un diamante de ese tamaño es creíble, o es un error de captura?
- ? ¿Qué harían con esas filas — borrarlas, corregirlas, preguntarle al dueño del dato? ¿Y qué le dirían?

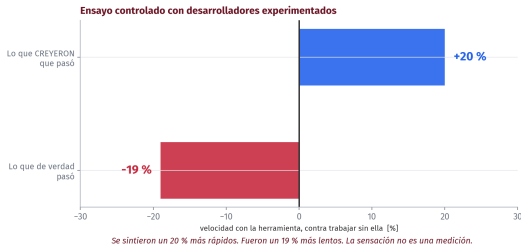
Reto 3 · Puesta en Común

La más importante de las tres:

- ¿Cuántas filas **físicamente imposibles** encontró cada mesa? ¿Todas encontraron las mismas?
- ¿Qué decidieron hacer con ellas: borrar, corregir, preguntar? No hay una sola respuesta correcta — pero sí hay que poder **defender** la que eligieron.
- La de fondo: si nadie hubiera revisado y se entrena un modelo de precios con esos datos, ¿qué pasa? ¿Quién se da cuenta, y cuándo?

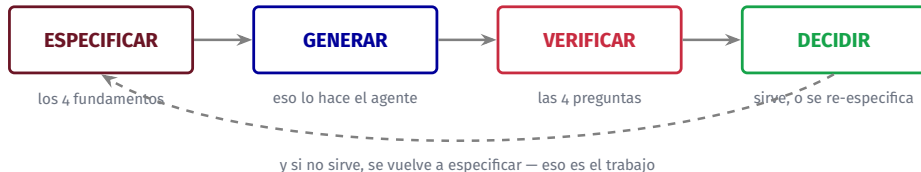
Hoy encontraron ustedes la trampa que en la Demo 3 encontré yo. Esa es exactamente la habilidad que este módulo quiere dejarles.

Y Cuidado con la Sensación



Ensayo controlado con desarrolladores experimentados: se sintieron un **20 % más rápidos** y fueron un **19 % más lentos**. No es un argumento contra la herramienta — es uno a favor de **medir**.

El Ciclo Real, y la Lista de Verificación



Las **cuatro preguntas** de VERIFICAR, para todo lo que devuelva un agente:

1. ¿Cuántas filas entraron y cuántas salieron?
2. ¿Hay ceros, vacíos o números redondos sospechosos?
3. ¿El resultado tiene sentido físico o de negocio?
4. ¿Qué decisión se toma con esto, y qué pasa si está mal?

Ninguna exige saber programar. Las cuatro exigen saber del negocio.

De la Gráfica a la App

Que otro lo pueda usar sin llamarlos

98 – 118 min

El Salto que Falta



El salto que separa un análisis de una herramienta

Mientras el análisis viva en una celda, cada vez que alguien quiera ver otra cosa tiene que llamarlos. Una app se la mandan y se acabó.

Qué Es Gradio, y Qué Es el Link

Gradio convierte una función de Python en una página web con controles. Uno le dice «esta función recibe un día y devuelve un gráfico», y él arma la interfaz solo: el menú, el botón, el espacio del resultado.

No hay que saber nada de páginas web. Y en Colab hace algo más: con `share=True` genera un **link público** — una dirección que cualquiera puede abrir desde su celular, mientras el cuaderno de ustedes hace el trabajo por detrás.

LO QUE HAY QUE SABER DEL LINK

Vive **mientras el cuaderno esté corriendo**. Si cierran Colab o la sesión se desconecta, deja de responder. Perfecto para una reunión o una demostración; **no** es un sistema permanente — eso se ve más adelante en el módulo.

La analogía honesta: es una extensión telefónica hacia su cuaderno, no una oficina nueva.

Demo · La App Más Chica que Sirve

```
import gradio as gr

def ver_demanda(dia):
    datos = taxis[taxis.dia_nombre == dia]
    return grafico_demanda(datos)

demo = gr.Interface(
    fn=ver_demanda,
    inputs=gr.Dropdown(DIAS, label="Día"),
    outputs=gr.Plot(label="Demanda por hora"),
    title="Demanda de taxis por día")

demo.launch(share=True)
```

Tres piezas: la **función**, los **controles**, y `launch(share=True)`, que genera el **link público**.

Lo que hay que saber del link: vive mientras el cuaderno esté corriendo. Perfecto para una reunión; no es un sistema permanente.

✂ Su Turno · La App de SU Dato

🕒 18 min

Eliján **uno** de sus tres datasets (mpg, car_crashes o diamonds) y conviertan su análisis en una **app de Gradio con link**, para alguien que decide:

La app debe tener **al menos dos controles** (qué filtrar o comparar, lo deciden ustedes), mostrar **una gráfica que se entienda sola**, y **no caerse** si la combinación elegida no tiene datos.

Cómo saber si van bien:

- ? ¿Le escribieron al agente el criterio de aceptación — incluido el caso sin datos?
- ? Abran su propio link y traten de romperlo. ¿Sobrevive?
- ? ¿Se lo mandarían a su jefe tal como está? Si dudan, ¿qué le falta?

Al final, tres voluntarios abren su link en el proyector. Ese es el cierre de la clase.

Cierre

Lo Que Aprendimos Hoy

💡 Conceptos:

- ✓ Autocompletado, asistente y **agente** no son lo mismo
- ✓ Una gráfica es un **argumento**, no un adorno
- ✓ Los **cuatro fundamentos** de un encargo
- ✓ Un **cero perfecto** casi siempre es una ausencia
- ✓ Las **cuatro preguntas** de verificación

🔧 Herramientas:

- seaborn y sus datasets de práctica
- El agente Gemini del panel de Colab
- gradio y launch(share=True)
- La **plantilla de encargo** — la que más van a usar

EL NÚMERO DEL DÍA

1.812 y 0. Los viajes en efectivo, y los que tenían propina registrada. El agente no vio nada raro.

Los Resultados Negativos de Hoy

La regla del curso: al menos un resultado negativo se dice en voz alta. Hoy hubo dos, medidos:

- ✗ **El agente entregó un análisis correcto con una conclusión falsa**, sin ninguna señal de alarma. El código estaba bien; el dato mentía.
- ✗ **Desarrolladores experimentados fueron 19 % más lentos** con estas herramientas en un ensayo controlado, sintiéndose 20 % más rápidos.

Ninguno es razón para no usarlas. Los dos son razón para medir en vez de confiar en la sensación — lo mismo que venimos haciendo todo el diplomado.

Si Se Llevan Una Sola Cosa

El agente escribe el código. Ustedes responden por el resultado.

Y eso no es una carga: es la razón por la que su criterio vale más ahora que hace tres años. Escribir se volvió barato; saber si el resultado tiene sentido, no.

La habilidad de hoy no es «usar Gemini». Es **especificar bien y verificar siempre** — y sirve con cualquier herramienta que salga el año que viene.

Nadie escribió una línea de código hoy. Y todos se van con una aplicación funcionando y tres ejercicios resueltos por ustedes mismos.

Adelanto · Qué No Se Pega en un Chat de IA

Hoy trabajamos con datos públicos de práctica. Su proyecto será con datos **de su empresa** — y ahí aplican tres reglas, desde ya:

- 🔒 **El dato confidencial no se pega.** Producción real, presiones, coordenadas, nombres de pozos o de clientes: nada de eso entra a un chat de IA sin autorización.
- ✓ **El esquema sí.** Los *nombres de las columnas* y los tipos — sin los valores — suelen bastar para que el agente escriba el código. Se corre localmente sobre el dato real.
- ? **En la duda, pregunten** a su área de seguridad de la información antes, no después.

Los proveedores de estos servicios pueden retener lo que se les pega — prompts y datos — por meses, y personas pueden revisarlo. En la Clase 3 esto se ve completo, con la letra chica en pantalla.

Lo Que Sigue, y Su Proyecto

- ➔ **Clase 2 — Que responda, y que no mienta.** Servir un modelo de verdad: qué entra, qué sale, y el error que funciona en el cuaderno y falla en producción.
- ➔ **Clase 3 — Los peligros, con nombre.** Qué se puede pegar en un chat de IA y qué no, librerías inventadas, y código que corre y está mal.
- ➔ **Clase 4 — Su proyecto,** presentado al grupo: su problema, su app, sus prompts, y un error del agente que ustedes cazaron.

✓ TAREA PARA LA PRÓXIMA

Equipo (máx. 3) y problema elegidos, escritos en dos frases: qué duele, y quién decidiría distinto con una herramienta. La entrega del proyecto es **hasta el jueves de la próxima semana.**

Datos y Referencias

-  **Adopción y confianza:** Stack Overflow Developer Survey y su análisis de la brecha de confianza (2026); *State of Code*, Sonar (enero de 2026).
-  **Productividad medida:** ensayo controlado de **METR**; informes de adopción de agentes 2026.
-  **Datos:** taxis, mpg, car_crashes y diamonds, de los conjuntos de práctica de **seaborn**. Una línea y están cargados.
-  Las figuras y las cifras de estas láminas salen de `figuras.py`. Las soluciones de los ejercicios reto **no existen en ningún archivo:** se construyen en clase.

Los logos pertenecen a sus marcas y se usan con fines educativos.

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

Módulo 7 · Clase 1 · SLB Ecuador · UDLA · 2026